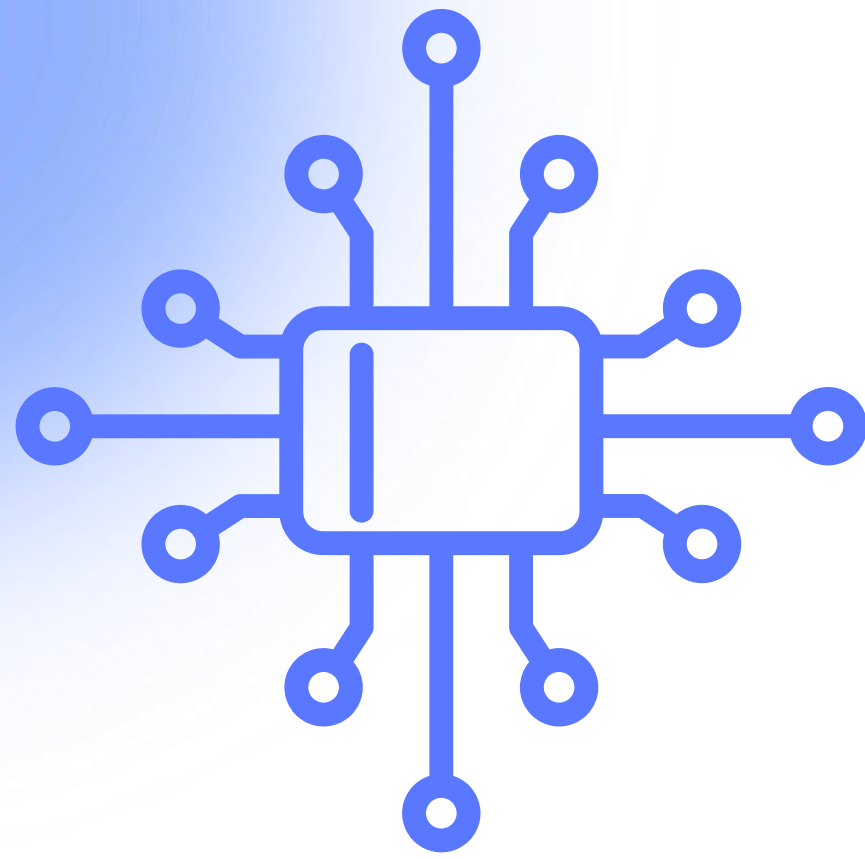
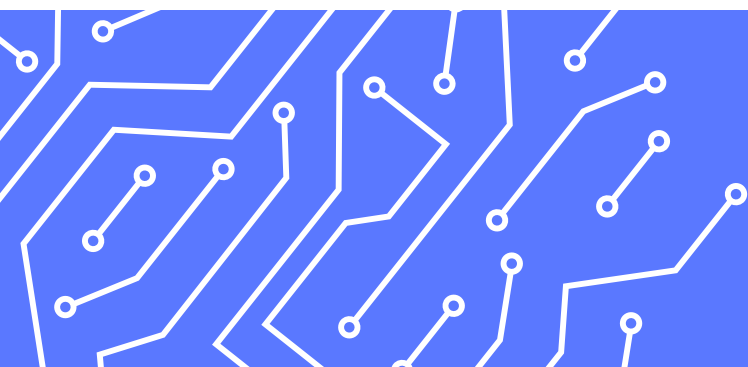
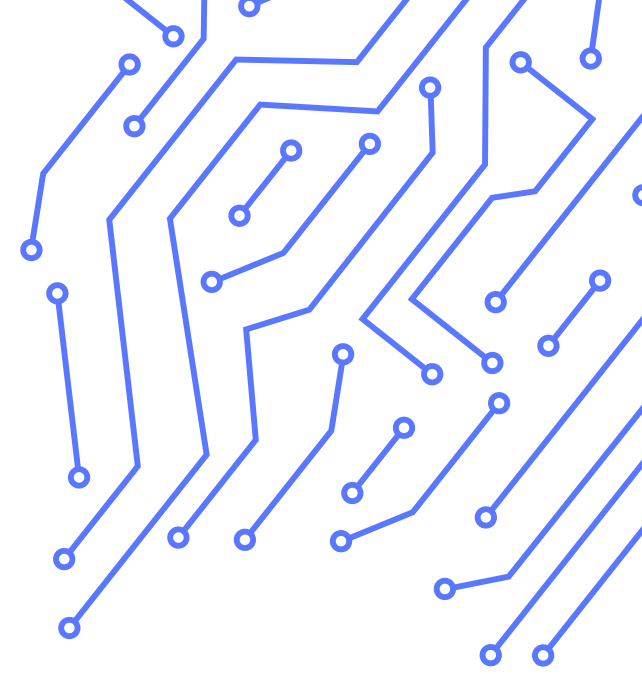




TRAINING AND REGULARIZATION

**MADE BY:
ALI RAAFAT**



TRAINING AND VALIDATION

we typically split the data into three main parts:

a) Training Set

- This is the data the model actually learns from (it computes weights and biases based on it)
- Usually takes the largest portion of the data (60–80%)
- The model sees inputs and correct outputs and tries to minimize error through backpropagation

b) Validation Set

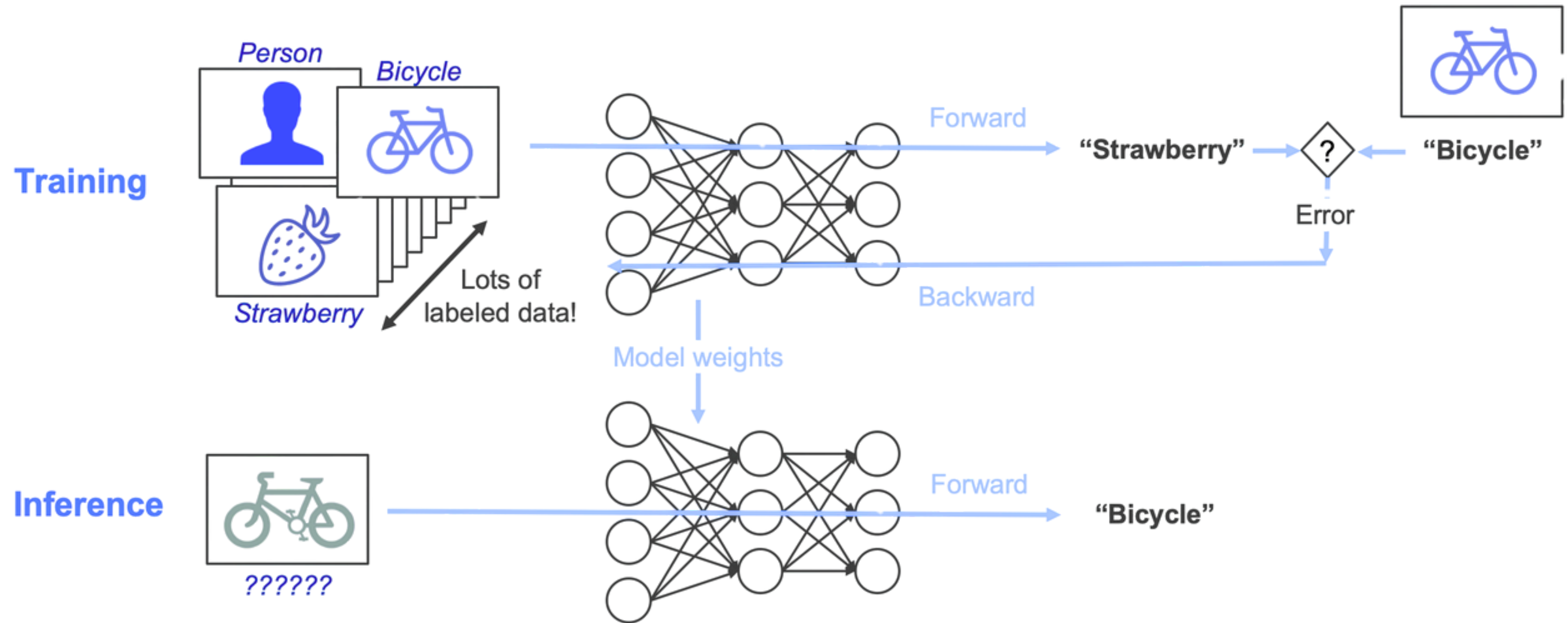
- Monitor the model's performance on unseen data
- Tune hyperparameters (learning rate, number of layers, batch size, etc.)
- Decide when to stop training (Early Stopping)

c) Test Set

- Used to measure the model's real-world performance on data it has never seen at all



TRAINING AND VALIDATION



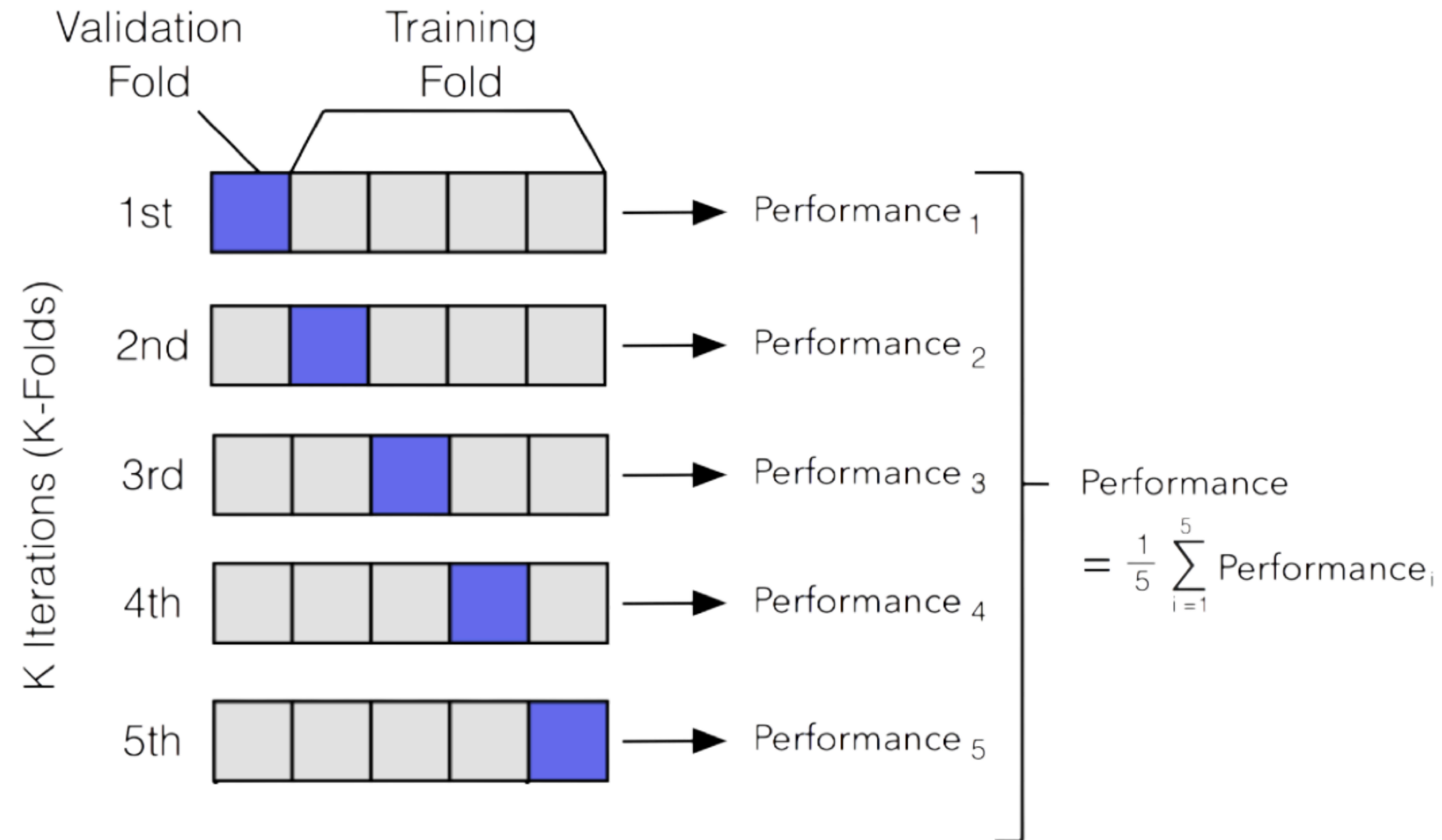
Clarification for Training Process

CROSS VALIDATION

Sometimes we use a technique called **K-Fold Cross Validation** instead of a fixed split.

The data is divided into **K parts**, and the model is trained **K times**, each time using a different part as validation.

This gives a more accurate evaluation, especially when data is limited.



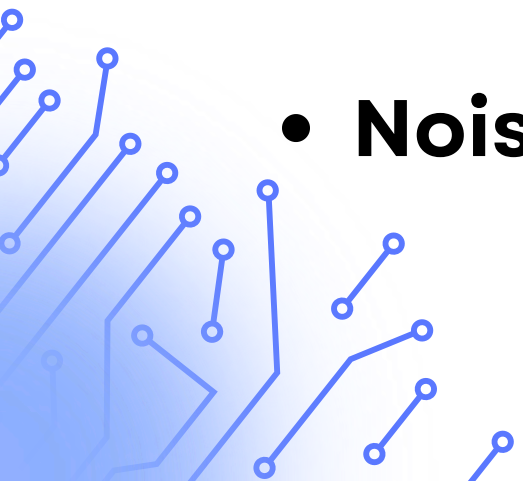
OVERFITTING

How to detect Overfitting

- Training Loss: keeps decreasing continuously
- Validation Loss: decreases at first, then starts increasing again

Causes of Overfitting

- **Insufficient training data** – not enough data for the model to learn a general pattern
- **Excessive model complexity** – too many layers or parameters relative to the problem size
- **Training for too long** (too many epochs)
- **Noise in the data** that the model starts learning as if it were a real pattern



REGULARIZATION

Regularization: A set of techniques used to prevent or reduce overfitting by adding constraints or penalties to the model, encouraging it to stay simple and general rather than memorizing fine details.

Main Types of Regularization:

a) L1 Regularization (Lasso)

- Adds a penalty to the loss function equal to the sum of the absolute values of the weights

$$Error = \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2 + \alpha \sum_{j=1}^m |w_j|$$

- Result: some weights become exactly zero, effectively performing a kind of feature selection



REGULARIZATION

b) L2 Regularization (Ridge / Weight Decay)

- Adds a penalty equal to the sum of squared weights

$$Error = \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2 + \alpha \sum_{j=1}^m w_j^2$$

- Result: weights shrink overall but rarely reach exactly zero, making the model more stable

c) Early Stopping

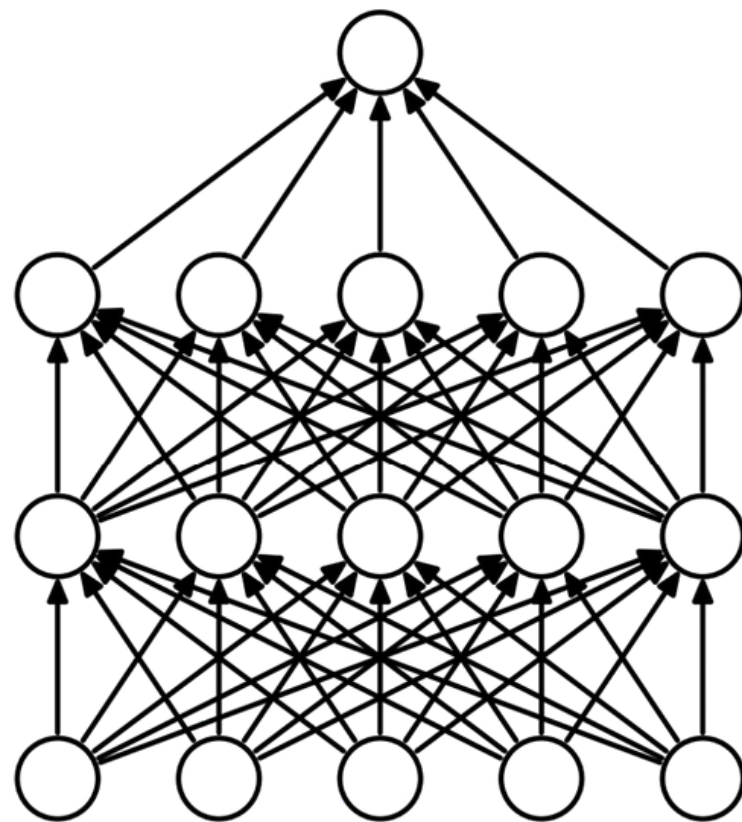
- As mentioned in overfitting: we monitor validation loss, and as soon as it starts increasing, we stop training immediately – even if more epochs remain



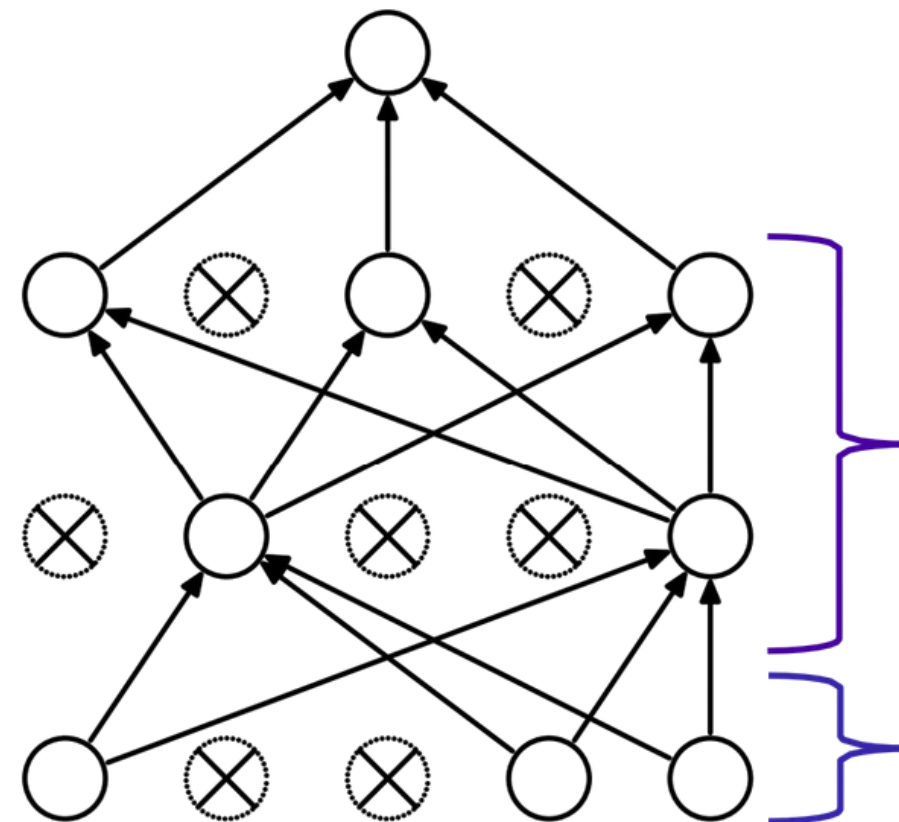
REGULARIZATION

d) Dropout

- A technique specific to Deep Learning
- During each training step, a random percentage of neurons (e.g., 20% or 50%) is temporarily disabled
- This prevents the network from relying too heavily on specific neurons, forcing it to learn more general and robust features



(a) Standard Neural Net



(b) After applying dropout.

e.g., drop 50% of
units in hidden layers

e.g., drop 20% of
units in input layers



REGULARIZATION

e) Data Augmentation

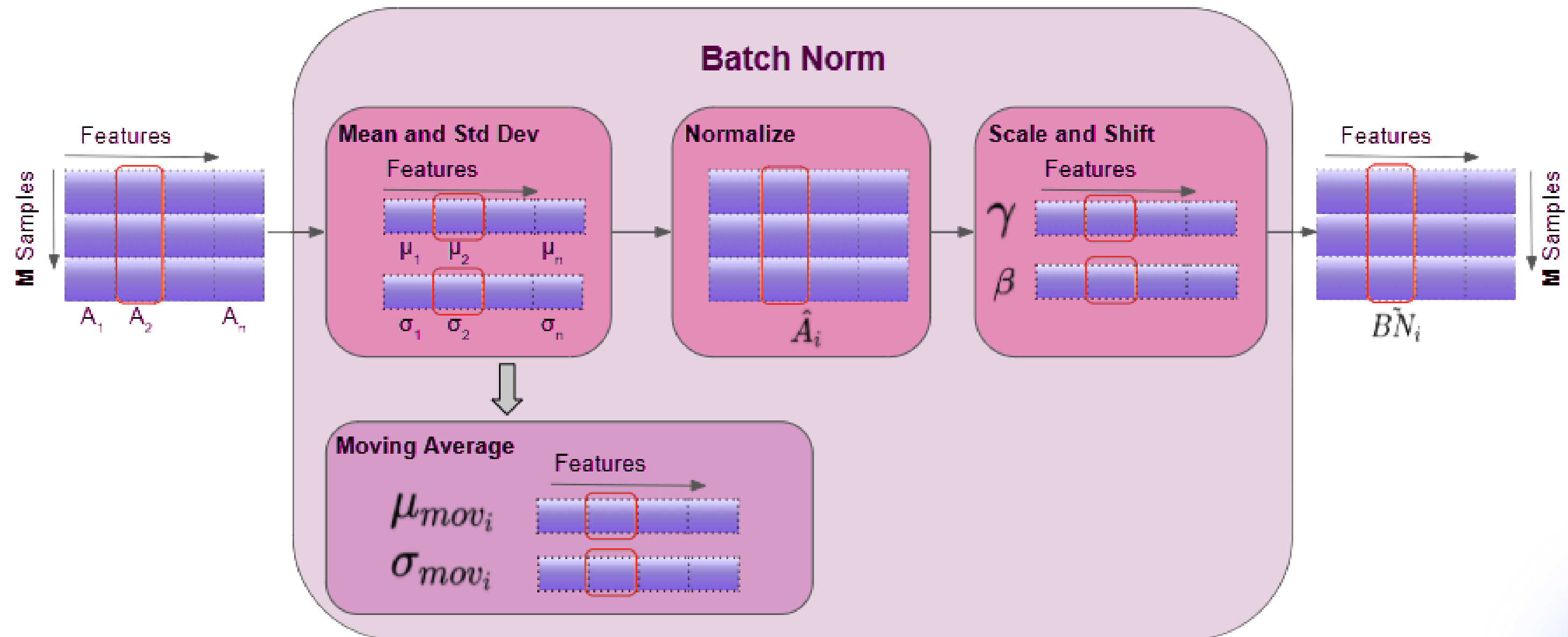
- Instead of reducing model complexity, we increase the diversity of the data itself
- By applying transformations to existing data (rotating images, flipping, changing brightness, etc.) to generate new, realistic variations
- This helps the model learn more general patterns instead of memorizing



REGULARIZATION

f) Batch Normalization

- Helps stabilize training by normalizing the inputs to each layer, which indirectly also helps reduce overfitting



THANK YOU

I hope you understand the
session well

Thanks for your attention